Système d'exploitation
CM6 : Entrées-sorties

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

mars 2026

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

Entrées-sorties bas niveau Unix

4 appels système fondamentaux d'Unix :

- `open` pour ouvrir un fichier (création d'une entrée dans la table des fichiers ouverts du processus)
- `write` pour écrire des octets dans le fichier ;
- `read` pour lire des octets depuis le fichier ;
- `close` pour le fermer.

Entrées-sorties standardisées

Par contraste, `stdio.h` utilise

- `fopen` pour ouvrir un fichier et créer un flux ;
- `fprintf`/`fscanf` pour des écritures/lectures de texte, formatées sur un flux ;
- `fwrite`/`fread` pour des écritures/lectures de binaire sur un flux ;
- `fclose` pour fermer un flux.
- `printf` et `scanf` correspondent à `fprintf` et `fscanf` sur la sortie standard et l'entrée standard.

En principe, ces fonctions fonctionnent quel que soit le système sur lequel est installé le compilateur C.

Entrées-sorties

- Les appels primitifs Unix (`read`, `write`, ...) laissent une grande liberté au programmeur mais aussi plus de travail.
- Les appels standards réduisent la liberté, mais offrent des services supplémentaires (portabilité, formatage, *bufferisation*).
- La *bufferisation* réduit le nombre d'appels systèmes utilisés et améliore la performance, mais entraîne des complications supplémentaires.

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

Fichier ouvert, descripteur de fichier

- Le système maintient pour chaque processus une *table des fichiers ouverts*.
- À chaque fichier ouvert par le processus est associé un petit entier (son indice dans cette table) appelé *descripteur de fichier* (*file descriptor*).
- Ce descripteur de fichier est ensuite donné en argument à tous les appels systèmes qui permettent au processus d'interagir avec lui.
- Le système maintient également la position de *la tête de lecture ou écriture* dans le fichier.

open

```
#include <fcntl.h>
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

- `pathname` : chemin (relatif ou absolu) dans l'arborescence des fichiers ;
- `flags` :
  - `O_RDONLY` pour lecture seule
  - `O_WRONLY` pour écriture seule
  - `O_RDWR` pour lecture et écriture (plus rare)

  Ces `flags` peuvent être « ou-bit-à-bit-és »avec des attributs de création ou d'états (voir plus loin).
- `mode` : dans le cas où il faut créer le fichier, donne les permissions (avant application du masque utilisateur `umask`) associées au fichier à créer.

Exemple : ouverture d'un fichier en lecture (1)

```
#include <fcntl.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    int fd;
    fd = open(argv[1], O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    } else {
        printf("fichier %s ouvert avec succès, descripteur %d\n",
                argv[1], fd);
    }
    return 0;
}
```

Exemple : ouverture d'un fichier en lecture (2)

Toujours tester la valeur de retour d'`open`.

Exemple : ouverture d'un fichier en lecture (2)

Toujours tester la valeur de retour d'`open`.

```
$ ./a.out open_exemple1.c
fichier open_exemple1.c ouvert avec succès, descripteur 3
$ ./a.out open_exemple1.C
open: No such file or directory
$ chmod u-r open_exemple1.c
$ ./a.out open_exemple1.c
open: Permission denied
$ ./a.out
open: Bad address
```

Exemple : Ouverture d'un fichier en écriture

```c
#include <fcntl.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int fd;
    /* O_CREAT : créer le fichier s'il n'existe pas
       O_TRUNC : si le fichier existe, le vider
       0666 : constante octale, en binaire 110110110, rw-rw-rw-
       mais on doit enlever aussi les bits de permissions de l'umask */
    fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0666);
    if (fd < 0) {
        perror("open");
        return 1;
    } else {
        printf("fichier %s ouvert avec succès, descripteur %d\n",
                argv[1], fd);
    }
    return 0;
}
```

Quelques états pour une ouverture `O_WRONLY` ou `O_RDWR`

- `O_TRUNC` : si le fichier existe, le vider
- `O_APPEND` : toutes les écritures se font à la fin du fichier
- `O_CREAT` : si le fichier n'existe pas, le créer (et utiliser le 3ème argument pour les permissions)
- `O_EXCL` : avec `O_CREAT`, si le fichier existe, échouer

Permissions

- Dans le cas d'un fichier potentiellement créé (bit `O_CREAT` présent dans l'état d'ouverture)
- préciser les permissions associées au nouveau fichier, dans le 3ème argument de `open`.
- Ceci peut se faire avec des drapeaux : `S_I<perms><pour-qui>` par exemple

  `S_IRUSR` l'utilisateur propriétaire (`USR`) a la permission de lire le fichier (`R`)

  `S_IRWXU` l'utilisateur propriétaire (`U`) a les permissions de lire (`R`), modifier (`W`) et exécuter (`X`) le fichier

  `S_IWGRP` le groupe propriétaire (`GRP`) a la permission de modifier le fichier

  `S_IRWXG` le groupe propriétaire (`G`) a la permission de lire, modifier et exécuter le fichier

  `S_IRWXO` les autres (`O`) ont la permission de lire, modifier et exécuter le fichier

  `S_IROTH` les autres (`OTH`) ont la permission de lire le fichier
- Ces drapeaux peuvent être « ou-bit-à-bit-és », par exemple `S_IROTH | S_IRGRP | S_IRUSR | S_IWUSR`

Permissions

- On peut aussi, alternativement, utiliser une constante en octal (base 8).
- En C, les constantes (entières) en octal commencent par un `0`.
- `r` vaut 4, `w` vaut 2 et `x` vaut 1, et il y a 3 types d'utilisateurs (utilisateur, groupe et autres), donc 3 chiffres en octal.
- Par exemple, `0644` signifie

Permissions

- On peut aussi, alternativement, utiliser une constante en octal (base 8).
- En C, les constantes (entières) en octal commencent par un `0`.
- `r` vaut 4, `w` vaut 2 et `x` vaut 1, et il y a 3 types d'utilisateurs (utilisateur, groupe et autres), donc 3 chiffres en octal.
- Par exemple, `0644` signifie lecture et écriture pour le propriétaire, lecture pour le groupe et lecture pour les autres.

Permissions

- On peut aussi, alternativement, utiliser une constante en octal (base 8).
- En C, les constantes (entières) en octal commencent par un `0`.
- `r` vaut 4, `w` vaut 2 et `x` vaut 1, et il y a 3 types d'utilisateurs (utilisateur, groupe et autres), donc 3 chiffres en octal.
- Par exemple, `0644` signifie lecture et écriture pour le propriétaire, lecture pour le groupe et lecture pour les autres.
- Pour dire lecture et exécution pour tous et en plus, modification pour le propriétaire, cela donne :

Permissions

- On peut aussi, alternativement, utiliser une constante en octal (base 8).
- En C, les constantes (entières) en octal commencent par un `0`.
- `r` vaut 4, `w` vaut 2 et `x` vaut 1, et il y a 3 types d'utilisateurs (utilisateur, groupe et autres), donc 3 chiffres en octal.
- Par exemple, `0644` signifie lecture et écriture pour le propriétaire, lecture pour le groupe et lecture pour les autres.
- Pour dire lecture et exécution pour tous et en plus, modification pour le propriétaire, cela donne : `0755`.
- Sauf que rien n'est simple en ce bas-monde ! Le masque utilisateur joue aussi un rôle.

Le masque utilisateur (umask)

- Un utilisateur a toujours un « masque de permissions ».

```
$ umask
0022
$ touch plop; mkdir hop; ls -ld plop hop
drwxr-xr-x. 1 pierre pierre 0 16 mars  18:06 hop
-rw-r--r--. 1 pierre pierre 0 16 mars  18:06 plop
```

- Par défaut, les fichiers normaux sont créés avec le mode 0666 (*the number of the beast*) auquel on enlève l'umask.
- Pour les répertoire c'est avec le mode 0777 auquel on enlève l'umask.
- On peut le modifier depuis la ligne de commande (pour le shell courant et ses descendants).

```
$ rm -r plop hop
$ umask 0001
$ touch plop; mkdir hop; ls -ld plop hop
drwxrwxrw-. 1 pierre pierre 0 16 mars  18:09 hop
-rw-rw-rw-. 1 pierre pierre 0 16 mars  18:09 plop
```

Le masque utilisateur (umask)

Les permissions de l'umask sont également retranchées dans le mode d'un appel à open qui crée un fichier !

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
int main(int argc, char *argv[])
{
    int fd[argc - 1];
    for (int i = 1; i < argc; i++)
        if ((fd[i - 1] = open(argv[i], O_WRONLY | O_CREAT, 0777)) < 0)
            perror("open");
    return 0;
}
```

```
$ cc mon_touch.c
$ ./a.out a b c
$ ls -l ?
-rwxr-xr-x. 1 pierre pierre 0 16 mars  18:19 a
-rwxr-xr-x. 1 pierre pierre 0 16 mars  18:19 b
-rwxr-xr-x. 1 pierre pierre 0 16 mars  18:19 c
```

Appel système umask

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
int main(int argc, char *argv[])
{
    int fd[argc - 1];
    mode_t old_mask = umask(0);
    printf("Ancien umask : %o\n", old_mask);
    for (int i = 1; i < argc; i++)
        if ((fd[i - 1] = open(argv[i], O_WRONLY | O_CREAT, 0777)) < 0)
            perror("open");
    return 0;
}
```

```
$ cc yolo_touch.c
~/depots/sr/cm/cm06 (main)$ ./a.out a b c
Ancien umask : 22
~/depots/sr/cm/cm06 (main)$ ls -l ?
-rwxrwxrwx. 1 pierre pierre 0 16 mars  18:21 a
-rwxrwxrwx. 1 pierre pierre 0 16 mars  18:21 b
-rwxrwxrwx. 1 pierre pierre 0 16 mars  18:21 c
```

Canaux standards

Trois fichiers déjà ouverts.

- Entrée standard, de descripteur 0, normalement reliée au clavier ;
- Sortie standard, de descripteur 1, normalement reliée au terminal ;
- La sortie standard des erreurs, de descripteur 2, normalement reliée au terminal.

close

```
#include <unistd.h>
int close(int fd);
```

- `close` ferme un fichier, c'est-à-dire libère l'entrée numéro `fd` dans la table des fichiers ouverts
- et libère les ressources associées.

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

read

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

- `ssize_t` : *signed size type*, un type entier assez grand, signé pour pouvoir retourner des valeurs négatives synonymes d'erreurs
- `fd` : le descripteur de fichier du fichier dans lequel on lit des octets
- `buf` : adresse dans la mémoire du processus où l'on veut copier ces octets
- `count` : nombre maximal d'octets que l'on veut lire
- Valeur de retour : nombre d'octets effectivement lus, peut être inférieur à `count` si, par exemple, il reste moins de `count` octets à lire dans le fichier
- Une valeur de retour < 0 signale une erreur, la valeur de retour 0 signale la fin du fichier.

read, exemple pédagogique

```c
/* read_exemple1.c */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    ssize_t n;
    char buf[4];
    n = read(fd, buf, 4);
    printf("n = %ld\n", n);
    n = read(fd, buf, 4);
    printf("n = %ld\n", n);
    n = read(fd, buf, 4);
    printf("n = %ld\n", n);
    return 0;
}
```

`read`, exemple pédagogique

```
$ echo 'Lol !!' >lol.txt
$ gcc -Wall read_exemple1.c
$ ./a.out lol.txt
n = 4
n = 3
n = 0
```

- Après le premier `read`, la tête de lecture est sur le premier point d'exclamation.
- Il reste alors 3 octets à lire : `!!\n`
- Le deuxième appel système `read` retourne alors 3 et non 4.
- Le dernier appel `read` retourne 0 car la tête de lecture est à la fin du fichier.

`read` bloquant

- Dans certaines circonstances, `read` est bloquant : il n'y a pas encore d'octet à lire dans le fichier.
- Le processus est endormi,
- réveillé par le système lorsque des octets sont arrivés.
- C'est le cas pour la lecture sur le terminal, dans un *socket*, ou dans un tube (*pipe*).
- Dans ces conditions, les appels `read` ne peuvent avoir lieu sans une connaissance *a priori* de la forme des données à lire, c'est-à-dire du protocole d'échange des données.

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

write

```
#include <unistd.h>
ssize_t write(int fd, const void *buf, size_t count);
```

- `fd` : le descripteur de fichier du fichier dans lequel on écrit des octets
- `buf` : adresse dans la mémoire du processus du premier octet à écrire dans le fichier
- `count` : nombre maximal d'octets à écrire dans le fichier
- Valeur de retour : nombre d'octets effectivement écrits dans le fichier ; peut être (plus rarement que pour `read`) inférieur à `count` si, par exemple, le fichier est sur un disque dur presque plein, ou que le processus a été interrompu par un signal

write, exemple pédagogique

```c
/* write_exemple1.c */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0666);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    ssize_t n;
    char buf[] = "Lol !!\n";
    /* On ne veut pas écrire l'octet nul final dans le fichier. */
    n = write(fd, buf, strlen(buf));
    printf("n = %ld\n", n);
    close(fd);

    return 0;
}
```

write, exemple pédagogique

```
$ gcc -Wall write_exemple1.c
$ ./a.out plop
n = 7
$ cat plop
Lol !!
$ umask
0022
$ ls -l plop
-rw-r--r--. 1 pierre pierre 7  7 avril 07:13 plop
```

Un éditeur de texte

```c
/* super_text_editor.c */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#define BUFSZ 256
int main(int argc, char *argv[])
{
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0666);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    ssize_t n;
    char buf[256];
    while ((n = read(0, buf, 256)) > 0) {
        printf("%ld octets écrits dans %s\n", n, argv[1]);
        write(fd, buf, n);
    }
    close(fd);
    return 0;
}
```

super_text_editor

```
$ gcc -Wall super_text_editor -o ste
$ ste hello_world.c
#include <stdio.h>
19 octets écrits dans hello_world.c
int main() { printf("hello, world!\n"); }
42 octets écrits dans hello_world.c
$ cat hello_world.c
#include <stdio.h>
int main() { printf("hello, world!\n"); }
$ gcc -Wall hello_world.c
$ ./a.out
hello, world!
```

Entrées-sorties bas niveau, attention

- Les entrées-sorties avec `read` et `write` sont très élégantes et simples, mais
- ne s'intéressent absolument pas aux données lues/écrites qui ne sont que des octets ;
- en particulier, les octets nuls dans les chaînes de caractères n'ont rien de spécial pour ces fonctions.
- De plus, il est tout de même nécessaire dans certains cas de connaître le type du fichier (`socket`, `pipe`, terminal) dans lequel on `read`, car cela rend la lecture bloquante et le problème de l'arrêt de la lecture demande une connaissance *a priori* des données transmises.

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

stdio.h

- Sur un système Unix, `stdio.h` est une couche au-dessus des appels systèmes `read`, `write`, etc.
- On ne manipule pas de descripteur de fichier directement mais des pointeurs `FILE *`, où `FILE` est un type opaque, qui contient notamment
  - le descripteur de fichier
  - le type d'ouverture du fichier (lecture, écriture, ...)
  - un tampon (en anglais *buffer*), alloué sur le tas, permettant de minimiser le nombre de lectures et d'écritures.
  - (si pertinent) la position dans le fichier/dans le tampon.
- Les 3 flux `FILE *` déjà ouverts sont `stdout` (sortie standard), `stderr` (sortie standard des erreurs) et `stdin` (entrée standard).

Lire/écrire un caractère à la fois

```
int fgetc(FILE *stream);
int getchar(void); // stdin
int ungetc(int c, FILE *stream);
int fputc(int c, FILE *stream);
int putchar(int c); // stdout
```

- Remarquer `int`, au lieu de `char` à cause de la constante `EOF` qui vaut -1, et `char` qui est parfois signé, parfois non.
- et `ungetc` qui permet de faire comme si on « remet » un caractère lu dans le flux, très utile pour l'analyse lexicale.
- Terminologie : on *get* un caractère depuis un flux on *put* un caractère vers un flux.

Lire/écrire une ligne à la fois

```
char *fgets(char *s, int size, FILE *restrict stream);
int fputs(const char *restrict s, FILE *restrict stream);
int puts(const char *s); // stdout
```

- Avec `fgets`, lire au plus `size - 1` caractères ou jusqu'à la fin de la ligne (ou du fichier). Si un caractère de fin de ligne est lu, il est stocké dans la chaîne. Un caractère nul est écrit après le dernier caractère écrit dans `s`.

Lire/écrire des octets

```
size_t fread(void *ptr, size_t size, size_t nmemb,
             FILE *restrict stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb,
             FILE *restrict stream);
```

- `fread` lit au maximum `size` octets et les copie à l'adresse `ptr` ;
- `fwrite` écrit `size` octets depuis l'adresse `ptr`.

Entrées-sorties formatées

```
int printf(const char *restrict format, ...);
int fprintf(FILE *restrict stream,
            const char *restrict format, ...);
int scanf(const char *restrict format, ...);
int fscanf(FILE *restrict flux,
           const char *restrict format, ...);
```

Asynchronisme

- Les lectures et écritures (appels système `read` et `write`) sous-jacents n'ont en général pas lieu lors de l'appel des fonctions précédentes.
- Il y a 3 types de `FILE *` :
  - *fully buffered* : un tampon (*buffer*) de taille fixe est utilisé pour la lecture ou l'écriture. Par exemple pour la lecture, ce tampon pourrait n'être rempli que lorsque tous ses caractères ont été consommés par des `get`. C'est le cas par défaut pour les fichiers normaux.
  - *line buffered* : le tampon n'est écrit que lorsqu'il contient une fin de ligne. C'est le cas pour l'écriture sur le terminal (`stdout`).
  - *unbuffered* : les `put` et `get` sont écrits/lus directement. C'est le cas pour l'écriture sur la sortie standard des erreurs (`stderr`).

Asynchronisme, cas *fully buffered*

Dans le cas *fully buffered*,

- les `put` dans le flux ont lieu dans un *buffer* (par exemple, de 4096 octets)
- et les écritures physiques (appel système `write`) ont lieu *par bloc* (par exemple, quand le tampon est plein)
- ou à la fermeture du flux (ou la fin du processus).
- Pour de la lecture, le *buffer* est (alloué et) rempli avec un `read` au premier `get` sur le flux,
- puis de nouveau, si l'on a atteint la fin du *buffer* et qu'on continue à `get`.

Asynchronisme, cas *line buffered*

Dans le cas *line buffered*,

- les `put` dans le flux ont lieu dans un *buffer* (par exemple, de 1024 octets)
- et les écritures physiques (appel système `write`) ont lieu quand le tampon est plein ou qu'une fin de ligne (`'\n'`) est `put`
- ou à la fermeture du flux (ou la fin du processus).
- Il n'y a pas de *line buffering* pour de la lecture. Dans le cas d'un `get` ou `read` sur le terminal, c'est le *terminal driver* qui s'assure que les données sont transmises pour lecture seulement lorsqu'une fin de ligne est entrée.

fclose

- `fclose`, en plus de fermer le fichier
- écrit les caractères restants dans le tampon
- puis le libère.

Quelques façons de contrôler la *bufferisation*

- `setvbuf` permet de changer la *bufferisation* associée au flux `f`, par exemple avec

  ```
  setvbuf(f1, NULL, _IONBF, 0); // non bufferisé
  setvbuf(f2, NULL, _IOFBF, 0); // fully buffered
  setvbuf(f3, NULL, _IOLBF, 0); // line buffered
  ```

  (les arguments mis à `NULL` ou `0` permettent de choisir manuellement un `buffer`, mais c'est déconseillé).
- `int fflush(FILE *flux);` force l'écriture d'un tampon
- Pour la lecture dans le terminal, il est parfois utile de vider le tampon en lisant tous les caractères restants jusqu'à la fin de ligne (ou du fichier).

  ```
  int c;
  scanf("%c", &c); // lecture d'un caractère sans sauter les blancs
  // il reste des choses dans la ligne (au moins '\n')
  while ((c = getchar()) != '\n' && c != EOF)
      ;
  ```

Entrées-sorties, introduction

`open` et `close`

`read`

`write`

Entrées-sorties standard

Lire et écrire sur le terminal

Lignes d'entrée du terminal

Sur Linux, `man termios` nous dit :

> *La longueur maximale de ligne est 4096 caractères (y compris le caractère saut de ligne terminal) ; les lignes de plus de 4096 caractères sont tronquées. Après 4095 caractères, le traitement de l'entrée (...) se poursuit, mais les données entrées après 4095 caractères jusqu'à (mais sans l'inclure) un saut de ligne terminal sont abandonnées. Cela assure que le terminal peut toujours recevoir plus d'entrées jusqu'à ce que au moins une ligne puisse être lue.*

Mais cette limite de 4096 caractères dépend du système, donc ne pas se reposer dessus... Par exemple

```
char buffer[4096];
scanf("%s", buffer);
```

est sûr, sur Linux, mais pas forcément sur d'autres Unix. De plus, si l'entrée standard est redirigée, on ne lit plus sur le terminal...

Sorties sur le terminal

- Plus simple, c'est simplement *line-buffered*
- à ceci prêt qu'une lecture sur le terminal force l'écriture du tampon de sortie avant la lecture.
- Cela permet par exemple d'écrire :

```
int n;
// pas une ligne, donc devrait rester dans le tampon
printf("Entrer le nombre d'étoiles : ");
scanf("%d", &n); // lecture, on écrit d'abord le tampon
```